

Lab 02: Static and Dynamic Arrays

Pointers and Dynamic Memory Allocation

Student Name: Rao Jawad Ur Rehman

CMS ID: 546040

Course: CS: Data Structures & Algorithms

Class: BSCS-15D / BSCS-15E

Date: 15th September 2026

Department: Department of Computing

Task 1: Static 1D Array and Pointer Traversal

```
#include <iostream>

using namespace std;

int main() {
    int sales[5];
    int* p = sales;

    cout << "Enter sales for 5 days: ";
    for (int i = 0; i < 5; i++) {
        cin >> *(p + i);
    }

    int total = 0;
    cout << "\nSales values: ";
    for (int i = 0; i < 5; i++) {
        cout << *(p + i) << " ";
        total += *(p + i);
    }
    cout << "\nTotal sales: " << total << endl;

    // Add 2 to the third day's value (index 2)
    *(p + 2) += 2;

    int updatedTotal = 0;
    cout << "\nUpdated sales values: ";
    for (int i = 0; i < 5; i++) {
        cout << *(p + i) << " ";
        updatedTotal += *(p + i);
    }
    cout << "\nUpdated total sales: " << updatedTotal << endl;

    return 0;
}
```

Question 1.1: Pointer Expressions Analysis

Explain `p`, `&p`, `*p`, and `*(p + 2)`. State the valid indices and explain why `*(p + 5)` cannot be used to read an element.

- `p`: The pointer variable itself, storing the starting memory address of the array (the location of `sales[0]`).
- `&p`: The physical memory address on the stack where the pointer variable `p` is stored.
- `*p`: Dereferences the pointer to retrieve the value held at `sales[0]`.

- `*(p + 2)`: Advances the base address by 2 integer units and dereferences it, accessing the third element (`sales[2]`).
- Valid indices: 0, 1, 2, 3, and 4 (for a 5-element array).
- Why `*(p + 5)` cannot be used: A 5-element array spans indices 0 through 4. Offset +5 attempts to read outside the allocated block into unreserved memory, which triggers an out-of-bounds access and undefined behavior.

Output

```
Enter sales for 5 days: 10 20 30 40 50
```

```
Sales values: 10 20 30 40 50
```

```
Total sales: 150
```

```
Updated sales values: 10 20 32 40 50
```

```
Updated total sales: 152
```

Task 2: Dynamic 1D Array Allocation and Validation

```
#include <iostream>
#include <iomanip>

using namespace std;

int main() {
    int n;
    cout << "Enter number of students (1-10): ";
    cin >> n;

    if (n < 1 || n > 10) {
        cout << "Error: Number of students must be between 1 and 10." << endl;
        return 1;
    }

    int* marks = new int[n];

    cout << "Enter " << n << " marks (0-100): ";
    for (int i = 0; i < n; i++) {
        cin >> *(marks + i);
    }

    int total = 0;
    int passCount = 0;

    cout << "\nMarks: ";
    for (int i = 0; i < n; i++) {
        cout << *(marks + i) << " ";
        total += *(marks + i);
        if (*(marks + i) >= 50) {
            passCount++;
        }
    }

    double average = static_cast<double>(total) / n;

    cout << "\nTotal: " << total << endl;
    cout << "Average: " << average << endl;
```

```

    cout << "Pass count (>= 50): " << passCount << endl;

    delete[] marks;
    marks = nullptr;

    return 0;
}

```

Question 2.1: Storage Classes & Memory Lifetime

Explain why the pointer variable and the allocated elements are different objects in memory.

The pointer variable `marks` is an automatic local variable residing on the stack that stores a memory address. The array elements created with `new int[n]` are located dynamically on the heap. Because both objects occupy separate memory segments with distinct lifecycles, deleting the heap allocation does not erase the stack pointer variable, and discarding the pointer without calling `delete[]` leads to a memory leak.

Question 2.2: Compile-Time vs Runtime Array Sizing

Explain why entering `n` at runtime works with `new int[n]`, while `int marks[n];` is not a standard C++ fixed-size array declaration when `n` is a runtime variable.

Standard ISO C++ requires fixed-size stack arrays to have their dimensions fixed at compile time so stack-frame sizes can be pre-computed. In contrast, `new int[n]` delegates the memory sizing to the heap manager during runtime execution. Syntax like `int marks[n]` represents variable-length arrays (VLAs), which are non-standard compiler extensions in C++.

Output

```

Enter number of students (1-10): 4
Enter 4 marks (0-100): 40 50 70 81

Marks: 40 50 70 81
Total: 241
Average: 60.25
Pass count (>= 50): 3

```

Task 3: 2D Fixed-Size Array with Row Pointers

```

#include <iostream>

using namespace std;

int main() {
    int sales[2][3];
    int (*rowPtr)[3] = sales;

    // Input using nested loops and pointer notation
    for (int r = 0; r < 2; r++) {
        cout << "Enter 3 days sales for Branch " << (r + 1) << ": ";
        for (int c = 0; c < 3; c++) {
            cin >> (*(rowPtr + r) + c);
        }
    }

    // Display sales table

```

```

cout << "\nSales Table (Branches x Days):\n";
for (int r = 0; r < 2; r++) {
    cout << "Branch " << (r + 1) << ": ";
    for (int c = 0; c < 3; c++) {
        cout << (*(rowPtr + r) + c) << " ";
    }
    cout << "\n";
}

// Separate loop for branch totals
cout << "\nBranch Totals:\n";
for (int r = 0; r < 2; r++) {
    int branchTotal = 0;
    for (int c = 0; c < 3; c++) {
        branchTotal += (*(rowPtr + r) + c);
    }
    cout << "Branch " << (r + 1) << " Total: " << branchTotal << endl;
}

// Separate loop for day totals
cout << "\nDay Totals:\n";
for (int c = 0; c < 3; c++) {
    int dayTotal = 0;
    for (int r = 0; r < 2; r++) {
        dayTotal += (*(rowPtr + r) + c);
    }
    cout << "Day " << (c + 1) << " Total: " << dayTotal << endl;
}

return 0;
}

```

Question 3.1: Row Pointer Stride & Double Pointers

Explain what `rowPtr + 1` points to and why its step differs from the step of an `int*` pointing to one integer. Why should `int** p = sales;` not be used for a fixed-size 2D array?

The type of `rowPtr` is `int (*)[3]`, meaning it points to an entire 3-element row rather than a single integer. Adding 1 advances the pointer by the full stride of a row ($3 * \text{sizeof}(\text{int})$ bytes), positioning it at the start of `sales[1]`. A standard `int` only advances by $1 * \text{sizeof}(\text{int})$ bytes.

`int** p = sales` is invalid because a built-in 2D array is stored as a flat, contiguous sequence of integers without intermediate pointer arrays. Treating it as a pointer-to-pointer causes the program to treat raw integer values as memory addresses, leading to invalid reads and segmentation faults.

Output

```

Enter 3 days sales for Branch 1: 2 4 6
Enter 3 days sales for Branch 2: 1 3 5

Sales Table (Branches x Days):
Branch 1: 2 4 6
Branch 2: 1 3 5

Branch Totals:
Branch 1 Total: 12
Branch 2 Total: 9

```

Day Totals:
Day 1 Total: 3
Day 2 Total: 7
Day 3 Total: 11

Task 4: Dynamic 2D Array

```
#include <iostream>

using namespace std;

int main() {
    int rows, cols;
    cout << "Enter number of students (rows 1-10): ";
    cin >> rows;
    cout << "Enter number of subjects (columns 1-10): ";
    cin >> cols;

    if (rows < 1 || rows > 10 || cols < 1 || cols > 10) {
        cout << "Error: Rows and columns must be between 1 and 10." << endl;
        return 1;
    }

    // Allocate array of row pointers
    int** marks = new int*[rows];
    for (int r = 0; r < rows; r++) {
        *(marks + r) = new int[cols];
    }

    // Read marks using pointer notation
    for (int r = 0; r < rows; r++) {
        cout << "Enter " << cols << " marks for Student " << (r + 1) << ": ";
        for (int c = 0; c < cols; c++) {
            cin >> (*(marks + r) + c);
        }
    }

    // Display matrix
    cout << "\nMarks Matrix (Students x Subjects):\n";
    for (int r = 0; r < rows; r++) {
        cout << "Student " << (r + 1) << ": ";
        for (int c = 0; c < cols; c++) {
            cout << (*(marks + r) + c) << " ";
        }
        cout << "\n";
    }

    // Calculate totals and find highest student (ties keep first student)
    cout << "\nStudent Totals:\n";
    int bestTotal = -1;
    int topStudent = 1;

    for (int r = 0; r < rows; r++) {
        int studentTotal = 0;
        for (int c = 0; c < cols; c++) {
            studentTotal += (*(marks + r) + c);
        }
        cout << "Student " << (r + 1) << " Total: " << studentTotal << endl;
    }
}
```

```

        if (r == 0 || studentTotal > bestTotal) {
            bestTotal = studentTotal;
            topStudent = r + 1;
        }
    }

    cout << "\nTop Student: Student " << topStudent << " with Total: " << bestTotal << endl;

    // Deallocate memory
    for (int r = 0; r < rows; r++) {
        delete[] *(marks + r);
    }
    delete[] marks;
    marks = nullptr;

    return 0;
}

```

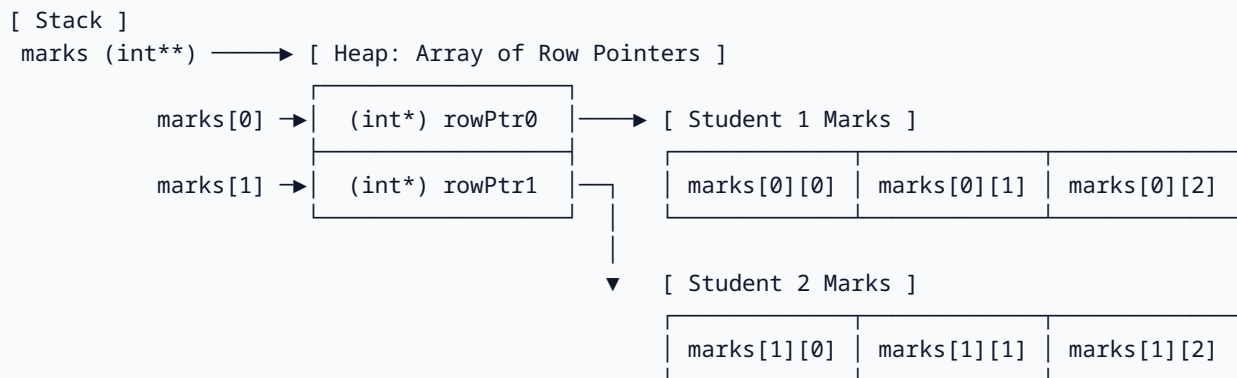
Question 4.1: Heap Memory Leaks & Deallocation Order

Explain what would remain allocated in memory if only `delete[] marks;` were executed.

Only the outer array of row pointers would be released from memory. The dynamically allocated integer rows for each student would remain orphaned on the heap without any surviving pointers to reference them, producing an irreversible memory leak.

Question 4.2: Memory Layout Architecture

For two students and three subjects, sketch the pointer variable, the array of two row pointers, and the two separately allocated rows. Label `marks`, `marks[0]`, and `marks[0][0]`.



Output

```

Enter number of students (rows 1-10): 2
Enter number of subjects (columns 1-10): 3
Enter 3 marks for Student 1: 50 60 70
Enter 3 marks for Student 2: 80 40 90

Marks Matrix (Students x Subjects):
Student 1: 50 60 70
Student 2: 80 40 90

Student Totals:
Student 1 Total: 180

```

Student 2 Total: 210

Top Student: Student 2 with Total: 210

Task 5: Dynamic Array Resizing by Copying

```
#include <iostream>

using namespace std;

int main() {
    int n;
    cout << "Enter initial number of marks (1-10): ";
    cin >> n;

    if (n < 1 || n > 10) {
        cout << "Error: Size must be between 1 and 10." << endl;
        return 1;
    }

    // Allocate initial block
    int* marks = new int[n];

    cout << "Enter " << n << " marks: ";
    for (int i = 0; i < n; i++) {
        cin >> *(marks + i);
    }

    // Step 1: Allocate a second block of size n + 1
    int* newMarks = new int[n + 1];

    // Copy original values into the new block using pointer notation
    for (int i = 0; i < n; i++) {
        *(newMarks + i) = *(marks + i);
    }

    // Read new mark into final position
    cout << "Enter new mark to append: ";
    cin >> *(newMarks + n);

    // Step 2: Release old block
    delete[] marks;

    // Update original pointer and size
    marks = newMarks;
    n = n + 1;
    newMarks = nullptr;

    // Display all values
    cout << "\nUpdated marks (" << n << " values): ";
    for (int i = 0; i < n; i++) {
        cout << *(marks + i) << " ";
    }
    cout << endl;

    // Step 3: Release final block exactly once
    delete[] marks;
    marks = nullptr;
```

```
    return 0;
}
```

Question 5.1: Dynamic Buffer Sizing & Reallocation

Explain why changing `n` alone does not increase the old block's capacity.

The variable `n` is merely a local integer and carries no automatic link to the heap's memory allocations. Once allocated, a dynamic array block has a fixed size assigned by the heap manager. Modifying the value of `n` does not extend the memory block; writing past the initial capacity corrupts neighboring heap memory. Proper resizing requires allocating a new, larger buffer, copying the data, and freeing the original block.

Output

```
Enter initial number of marks (1-10): 3
Enter 3 marks: 45 60 75
Enter new mark to append: 90

Updated marks (4 values): 45 60 75 90
```

Task 6: Code Inspection and Debugging

Question 6.1: Memory Bug Identification

Identify three errors in the given code fragment. For each, state the problem and explain its consequence.

1. Off-by-one loop condition (`i <= n`): The loop performs 4 iterations (indices 0 through 3) on an array sized for only 3 elements. This performs an illegal write past the allocated boundary, risking heap corruption.
2. Incorrect deallocation operator (delete values instead of `delete[] values`): Scalar delete is applied to an array created with `new[]`. This violates C++ memory requirements and leads to undefined behavior or allocator corruption.
3. Use-after-free access (`cout << values[0]` after delete): The program reads `values[0]` after the buffer has already been deallocated. Dereferencing this dangling pointer reads invalid or reclaimed memory.

Question 6.2: Corrected Program Implementation

Write a corrected complete program that reads and displays all three integers, releases the allocation correctly, and resets the pointer.

```
#include <iostream>

using namespace std;

int main() {
    int n = 3;
    int* values = new int[n];

    cout << "Enter 3 integers: ";
    // Error 1 fixed: Use i < n instead of i <= n to avoid out-of-bounds access
    for (int i = 0; i < n; i++) {
        cin >> *(values + i);
    }
}
```

```

// Error 3 fixed: Display values BEFORE deallocating the memory block
cout << "Values entered: ";
for (int i = 0; i < n; i++) {
    cout << *(values + i) << " ";
}
cout << endl;

// Error 2 fixed: Use delete[] instead of scalar delete to avoid undefined behavior
delete[] values;
values = nullptr;

return 0;
}

```

Question 6.3: Memory Concepts Distinction

Briefly distinguish an out-of-bounds access, a dangling pointer, and a memory leak. Explain why a program appearing to run successfully does not prove its memory accesses are valid.

- Out-of-bounds access: Reading from or writing to memory locations outside the allocated boundaries of an array, risking data corruption.
- Dangling pointer: A pointer variable that continues referencing a memory address after the target memory has already been freed.
- Memory leak: Dynamic heap memory that was never released with `delete[]` and can no longer be reached, consuming system memory indefinitely.
- Apparent success vs. validity: Raw pointers in C++ do not perform automatic runtime safety or boundary checks. Undefined behavior does not guarantee an immediate crash; an invalid operation may silently overwrite padding memory without surfacing an error immediately, but will produce unstable behavior across different compilers, machines, or memory loads.

Output

```

Enter 3 integers: 4 5 6
Values entered: 4 5 6

```